Graph Algorithms

Shortest path

All pairs shortest path problem

All pairs shortest path problem

- **Input**.
  - A directed graph $G = (V, E)$ with a weight on each edge
  - The edge weight could be negative, but there is not negative-weight cycle
- **Output**: The shortest path distance from u to v for all pairs of u,v $\in V$.

- **Brute-force solution.**
  - Apply Bellman-Ford on each node u $\in V$
  - **Runtime** = (n. mn) = $O(n^2m)$
  - **Floyd-Warshal** algorithm: $O(n^3)$

All pairs shortest path problem: DP: First solution

- Subproblem: Weight of shortest path from u to v using at most i edgess

$$D_i(u,v) = min \begin{cases} D_{i-1}(u,v) & \text{shortest path uses at most i-1 edges} \\ min_{(x,v)\in E}\{D_{i-1}(u,x) + w(x,v)\} & \text{shortest path uses exactly i edges} \end{cases}$$

$$D_0(u,u) = 0$$
$$D_0(u,v) = \infty \text{ where } u \neq v$$
$$D_0(u,v) = w(u,v) \text{ where } (u,v) \in E$$

Runtime: $O(n^4)$: for each i, for each node u, go through all edges. = $O(V.V.E) = O(V^4)$
Space: $O(n^3)$
you can store one 2-D array instead of all, therefore space could be reduced $O(n^3)$

All pairs shortest path problem: DP: Floyd-Warshal

- V = {1, 2, …, n}
- $D_k[i, j]$ = weight of shortest path from i to j using only vertices in {1…k} as intermediate vertices
  - Goal. finding **$D_n[i, j]$**
- **Base cases:**
  - $D_0[i, j]$: shortest path weight from i to j without using intermediate vertices
  - $D_0[i, j] = 0$ if i == j
  - $D_0[i, j] = w(i, j)$ if $(i, j) \in E$
  - $D_0[i, j] = \infty$ otherwise // shortest path from i to j without using any intermediate vertices

All pairs shortest path problem: DP: Floyd-Warshal

- To compute $P=D_k[i, j]$, using smaller subproblems, we know we have the solution to $D_{k-1}[i, j]$ and we want to use it to solve $D_k[i, j]$. There are two cases
  - Case 1: k is not used in P
    - Interior nodes are all in {1, …, k-1}
  - Case 2: k is used in P
    - The path from i to j has to go from i to k and then from k to j. Interior nodes on paths i to k and k to j are all in {1, …, k-1}

$$D_k[i, j] = \min \begin{cases} D_{k-1}[i, k] + D_{k-1}[k, j] & \text{use vertex k} \\ D_{k-1}[i, j] & \text{don't use vertex k} \end{cases}$$

All pairs shortest path problem: DP: Floyd-Warshal

```
for i from 0 to n
    for j from o to n
        if i == j
            D0[i, j] = 0
        else if (i, j) ∈ E
            D0[i, j] = w(i, j)
        else    D0[i, j] = ∞
for k from 0 to n-1 do
        for i from 1 to n do
                for j from 1 to n do
                        Dk[i, j] := min{Dk−1[i, j], Dk−1[i, k] + Dk−1[k, j]}
```

- Runtime: $O(n^3)$
- Space: $O(n^3)$
  - By storing **n** nXn arrays, and the original graph.
  - As with Bellman-Ford, we don't really need to store all n of the D_k
    - Space: $O(n^2)$

All pairs shortest path problem

```
Initialize D0[i, j] as before
for k from 0 to n-1 do
        for i from 1 to n do
                for j from 1 to n do
                        D[i, j] := min{D[i, j], D[i, k] + D[k, j]}
```

- Runtime: $O(n^3)$
- Space: $O(n^2)$

Applications of Shortest Path

Map routing

Robot navigation

Network routing protocols (OSPF, BGP, RIP)

Single-Source-Shortest Path in DAGs

Single-source shortest path in DAG

Shortest paths are always well defined in a DAG, Since there are no negative-weight cycle in a graph

- If the DAG contains a path from u to v, u precedes v in the topological sort
- If **u** comes before **v** in the topological order, there is no path from **v** to **u**

Single-source shortest path in DAG

DAG-Shortest-Paths(G, s)

Topologically sort the vertices of G

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

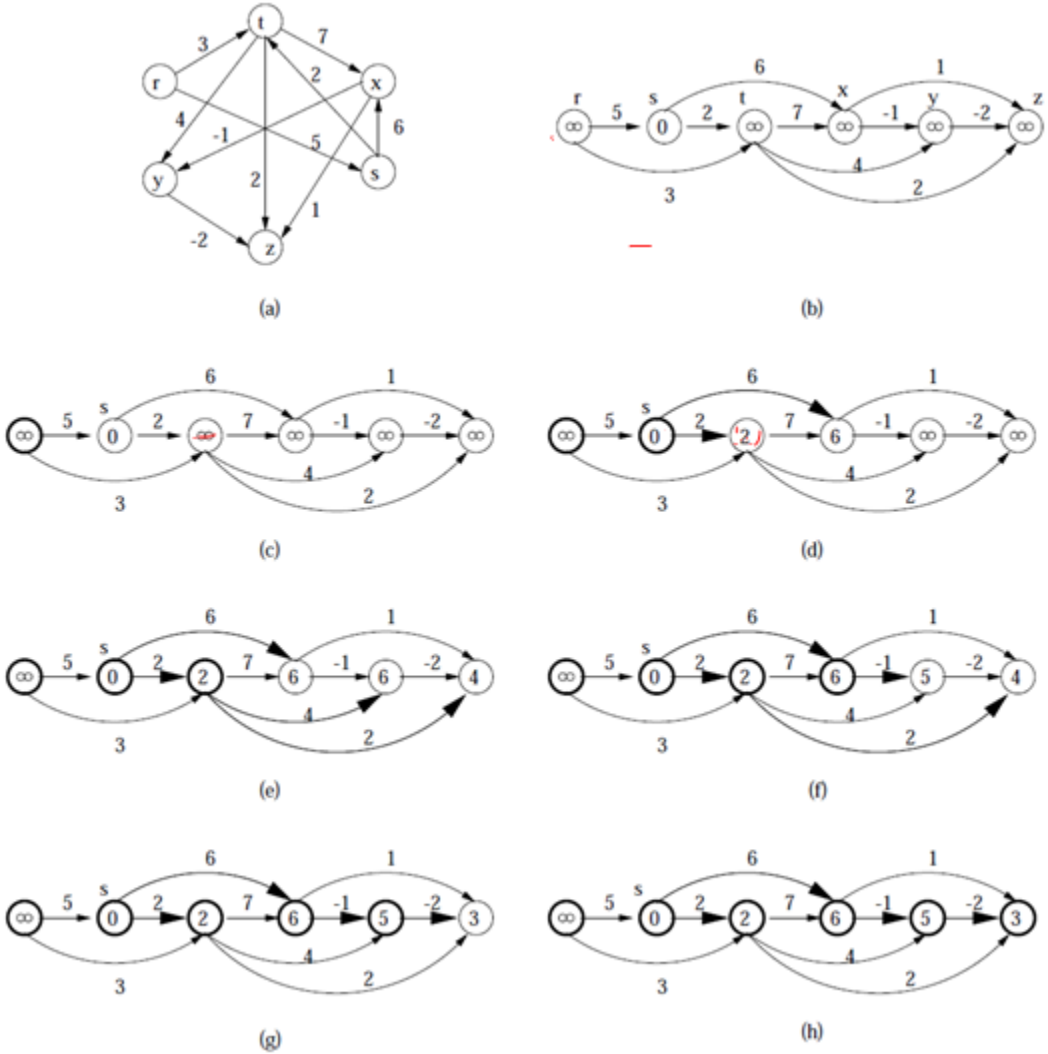
for each vertex u, taken in topologically sorted order

for each $v \in Adj[u]$

if $d[v] > d[u] + w(u, v)$

$d[v] \leftarrow d[u] + w(u, v)$

Single-source shortest path in DAG: Example



Single-source shortest path in DAG: Runtime: Θ(V+E)

DAG-Shortest-Paths(G, s)

Topologically sort the vertices of G

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$ } Θ(V)

do $d[v] \leftarrow \infty$

for each vertex u, taken in topologically sorted order } Θ(V+E)

for each $v \in Adj[u]$

if $d[v] > d[u] + w(u, v)$

$d[v] \leftarrow d[u] + w(u, v)$